**Alexandria University, Faculty of Engineering**
**Specialized Scientific Programs**
**Natural Language Processing**
**Instructor: Prof. Dr. Ayman Khalafallah**
**Assigned: 1/3/2026**
**Due: 10/3/2026**


## Assignment Overview

In this assignment, we work on two primary tasks:

1. Building a language model based on N-grams using an unsupervised subset of the IMDB reviews dataset. The model will be evaluated using the perplexity metric and the results will be analyzed.
2. Performing a Natural Language Inference (NLI) task using two different approaches: Naive Bayes and Logistic Regression. A confusion matrix will also be generated and performance metrics will be derived.

# 1 Part 1: N-gram Language Model

## 1.1 Objective

An N-gram language model is one of the earliest and simplest approaches to language modeling. In this part, the N-gram model is constructed step-by-step.

## 1.2 Dataset: WikiText

We will use the WikiText dataset from Hugging Face: `Salesforce/wikitext`. Specifically, we will use only the split `wikitext-2-raw-v1`.

## 1.3 Steps and Requirements

### 1.3.1 1. Data Preprocessing

- Tokenize the text into words.
- Apply normalization such as:
  - Lowercasing
  - Removing punctuation
  - any suitable preprocessing step

### 1.3.2 2. Constructing the N-gram Model

- Start with a unigram model (optional but recommended).
- Extend to bigrams, trigrams at least and any chosen N-gram size additionally.
- Implement Laplace smoothing to handle unseen N-grams.

### 1.3.3 3. Evaluation: Perplexity

- Train the model on the training set, then compute perplexity on the testing set.
- Compare different N-gram settings.

**Eng. Hana Ghanem**
**Eng. Mazen Yasser**
**Eng. Maria Bassem**
**Eng. Mohamed Salama**

#### 1.3.4 4. Saving and Inference

- Save the trained N-gram model parameters (e.g., vocabulary, N-gram counts, and smoothing configuration) to disk.
- Implement an inference function that:
  - Loads the saved model from disk.
  - Takes an input text prompt.
  - Generates the next tokens sequentially until the `</end>` token is produced or max number of tokens generated.

# 2 Part 2: Text Classification

## 2.1 Dataset: AG News

We will use the AG News dataset from Hugging Face: `sh0416/ag_news`.

This dataset is a news topic classification benchmark. Each example consists of a news **text** and a corresponding **label** indicating the topic category. The task is to predict the correct topic label for each news article.

## 2.2 Preprocessing for Classification

Each news text must be converted into a suitable representation for classification. The preprocessing includes:

- Text normalization (e.g., lowercasing and punctuation removal).
- Tokenization into words.

## 2.3 Naive Bayes Classification

### 2.3.1 Implementation From Scratch

Implement a Naive Bayes classifier using only NumPy:

- Compute class priors.
- Compute likelihood of tokens given each class.
- Predict labels for unseen text by maximizing the posterior probability.

### 2.3.2 Comparison with Scikit-learn

- Use `CountVectorizer` to transform text data into a bag-of-words representation.
- Use `MultinomialNB` for the classification step.
- Compare results with the from-scratch implementation using the evaluation metrics in Part 2.3.

## 2.4 Logistic Regression

### 2.4.1 Feature Representation

In this section, features are constructed using word bi-grams. For example, consider the sentence:

*"I love this movie very much"*

The word bi-grams are:

$$(I, love), (love, this), (this, movie), (movie, very), (very, much)$$

Each text is represented by a binary vector of length equal to the number of unique bi-grams in the dataset vocabulary, with a 1 if the bi-gram occurs in the text and a 0 otherwise.

#### 2.4.2 Implementation From Scratch

Implement Logistic Regression using NumPy:

- Initialize weights and bias.
- Train using (mini-)batch stochastic gradient descent.
- Minimize the cross-entropy loss.
- Shuffle the training data each epoch.

#### 2.4.3 Comparison with Scikit-learn

Compare performance with:

- `sklearn.linear_model.LogisticRegression`
- `sklearn.linear_model.SGDClassifier` (configured to behave like logistic regression).

### 2.5 Confusion Matrix and Evaluation Metrics

#### 2.5.1 Implementation From Scratch

1. Implement a function to generate a confusion matrix given predictions and ground-truth labels.
2. From this confusion matrix, compute:
   - Precision (per class and macro-averaged)
   - Recall (per class and macro-averaged)
   - F1-score (per class and macro-averaged)
3. Do not use any libraries except NumPy.

### 2.6 Comparison with Scikit-learn

Compare your confusion matrix and derived metrics with:

- `sklearn.metrics.confusion_matrix`
- `sklearn.metrics.precision_score`
- `sklearn.metrics.recall_score`
- `sklearn.metrics.f1_score`

## 3 Additional Notes

- Memory in Colab is limited (approximately 12GB).
- Use appropriate data types (prefer `float32` over `float64`).
- Free unused memory using the `del` command.
- Shuffle training data each epoch.
- Prefer vectorized operations instead of Python loops.

# 4 Team and Submission

## Team

The project should be completed in teams of three.

## Submission Requirements

Submit a ZIP file named:

`id1_id2_id3_asg1.zip`

The archive must include:

- Jupyter notebooks (.ipynb)
- Code for all parts
- Explanations and documentation
- Results and comparison plots/tables

**Eng. Hana Ghanem** **Eng. Maria Bassem**
**Eng. Mazen Yasser** **Eng. Mohamed Salama**